

Dokumentacja funkcjonalnosci - Home Budget

1. Struktura danych

Aplikacja wykorzystuje dwupoziomowa strukture danych z roznymi typami.

1.1 Model Transaction

Lokalizacja: `src/services/Api.ts`

```
interface Transaction {
  id: string; // string - unikalny identyfikator
  date: string; // string (format YYYY-MM-DD) - data transakcji
  amount: number; // number - kwota transakcji
  description: string; // string - opis transakcji
  categoryId: string; // string - referencja do obiektu Category
  (relacja)
}
```

1.2 Model Category

Lokalizacja: `src/services/Api.ts`

```
interface Category {
  id: string; // string - unikalny identyfikator
  name: string; // string - nazwa kategorii
  type: "income" | "expense"; // union type - typ kategorii
}
```

1.3 Stan aplikacji (Store State)

Lokalizacja: `src/flux/Store.ts`

```
interface StoreState {
  categories: Category[]; // array of objects - lista kategorii
  transactions: Transaction[]; // array of objects - lista
  transakcji
  monthFilter: string; // string - aktualny filtr miesiaca
  loading: {
    // object - stany ladowania
    categories: boolean;
    transactions: boolean;
  };
  error: {
    // object - komunikaty bledow
  };
}
```

```
categories: string | null;  
transactions: string | null;  
};  
}
```

1.4 Podsumowanie typow danych

Typ	Przykład uzycia
string	id, name, description, date, categoryId
number	amount
boolean	loading.categories, loading.transactions
object	loading, error, Category, Transaction
array	categories[], transactions[]
array of objects	Category[], Transaction[]
union type	"income" / "expense", "button" / "submit" / "reset"
Date (jako string)	date w formacie YYYY-MM-DD

2. Funkcjonalnosci CRUD

2.1 Dodawanie danych

Dodawanie transakcji

- Opis: Uzytkownik moze dodac nowa transakcje poprzez formularz
- Lokalizacja formularza: `src/components/transactions/TransactionForm.tsx`
- Lokalizacja strony: `src/pages/TransactionAdd.tsx` lub `src/pages/TransactionList.tsx`
- Akcja Flux: `actions.addTransaction()`
- Dane wejsciowe: date, amount, description, categoryId

Dodawanie kategorii

- Opis: Uzytkownik moze dodac nowa kategorie
- Lokalizacja formularza: `src/components/categories/CategoryForm.tsx`
- Lokalizacja strony: `src/pages/CategoryAdd.tsx` lub `src/pages/CategoryList.tsx`
- Akcja Flux: `actions.addCategory()`
- Dane wejsciowe: name, type

2.2 Edycja danych

Edycja transakcji

- Opis: Uzytkownik moze edytowac istniejaca transakcje
- Lokalizacja: `src/pages/TransactionList.tsx`
- Akcja Flux: `actions.editTransaction(id, updates)`
- Przycisk edycji w: `src/components/transactions/TransactionItem.tsx`

Edycja kategorii

- Opis: Uzytkownik moze edytowac istniejaca kategorie
- Lokalizacja: `src/pages/CategoryList.tsx`
- Akcja Flux: `actions.editCategory(id, updates)`

- Przycisk edycji w: `src/components/categories/CategoryItem.tsx`

2.3 Usuwanie danych

Usuwanie transakcji

- Opis: Użytkownik może usunąć transakcję z potwierdzeniem
- Lokalizacja: `src/pages/TransactionList.tsx`
- Akcja Flux: `actions.deleteTransaction(id)`
- Potwierdzenie: `window.confirm()`

Usuwanie kategorii

- Opis: Użytkownik może usunąć kategorię z potwierdzeniem
- Lokalizacja: `src/pages/CategoryList.tsx`
- Akcja Flux: `actions.deleteCategory(id)`
- Potwierdzenie: `window.confirm()`

2.4 Sortowanie

Sortowanie transakcji

Lokalizacja: `src/pages/TransactionList.tsx`

Dostępne kierunki sortowania (4 opcje):

1. date-desc - Data: najnowsze pierwsze
2. date-asc - Data: najstarsze pierwsze
3. amount-desc - Kwota: największa pierwsza
4. amount-asc - Kwota: najmniejsza pierwsza

Implementacja:

```
const sortedTransactions = [...state.transactions].sort((a, b) => {
  switch (sortType) {
    case "date-desc":
      return new Date(b.date).getTime() - new
Date(a.date).getTime();
    case "date-asc":
      return new Date(a.date).getTime() - new
Date(b.date).getTime();
    case "amount-desc":
      return b.amount - a.amount;
    case "amount-asc":
      return a.amount - b.amount;
  }
});
```

Sortowanie kategorii

Lokalizacja: `src/pages/CategoryList.tsx`

Dostępne kierunki sortowania (2 opcje):

1. asc - Alfabetycznie A-Z
2. desc - Alfabetycznie Z-A

Implementacja:

```
const sortedCategories = [...state.categories].sort((a, b) => {
  return sortOrder === "asc"
    ? a.name.localeCompare(b.name, "pl")
```

```
      : b.name.localeCompare(a.name, "pl");
    });
```

2.5 Filtrowanie

Filtr miesięczny

- Lokalizacja: `src/pages/Dashboard.tsx`
- Opis: Filtrowanie transakcji według wybranego miesiąca
- Akcja Flux: `actions.setMonthFilter(month)`
- Format: YYYY-MM (np. "2026-01")

3. Komunikacja z API

Aplikacja komunikuje się z JSON Server poprzez bibliotekę Axios.

3.1 Konfiguracja

Lokalizacja: `src/services/Api.ts`

```
const api = axios.create({
  baseURL: "http://localhost:3001",
  headers: {
    "Content-Type": "application/json",
  },
});
```

3.2 Endpointy kategorii

Operacja	Metoda HTTP	URL	Przekazywane dane	Zwracane dane
Pobierz wszystkie	GET	/categories	brak	Category[]
Utworz	POST	/categories	{ id, name, type }	Category
Aktualizuj	PATCH	/categories/:id	{ name?, type? }	Category
Usun	DELETE	/categories/:id	brak	void

3.3 Endpointy transakcji

Operacja	Metoda HTTP	URL	Przekazywane dane	Zwracane dane
Pobierz wszystkie	GET	/transactions	brak	Transaction[]
Pobierz jedna	GET	/transactions/:id	brak	Transaction
Utworz	POST	/transactions	{ id, date, amount, description, categoryId }	Transaction

Aktualizuj	PATCH	/transactions/:id	{ date?, amount?, description?, categoryId? }	Transaction
Usun	DELETE	/transactions/:id	brak	void

3.4 Przykłady implementacji

GET - Pobranie wszystkich kategorii

```
async all(): Promise<Category[]> {
  const response = await api.get<Category[]>("/categories");
  return response.data;
}
```

POST - Utworzenie transakcji

```
async create(transaction: Omit<Transaction, "id">):
Promise<Transaction> {
  const newTransaction = {
    ...transaction,
    id: Date.now().toString(),
  };
  const response = await api.post<Transaction>("/transactions",
newTransaction);
  return response.data;
}
```

PATCH - Aktualizacja kategorii

```
async update(id: string, updates: Partial<Omit<Category, "id">>):
Promise<Category> {
  const response = await api.patch<Category>(`/categories/${id}`,
updates);
  return response.data;
}
```

DELETE - Usuniecie transakcji

```
async delete(id: string): Promise<void> {
  await api.delete(`/transactions/${id}`);
}
```

4. Walidacja formularzy

4.1 Walidacja formularza transakcji

Lokalizacja: `src/components/transactions/TransactionForm.tsx`

Pole	Warunek walidacji	Komunikat błędu
date	Pole nie może być puste	"Data jest wymagana"
date	Data nie może być w przyszłości	"Data nie może być w przyszłości"
amount	Pole nie może być puste lub NaN	"Kwota musi być liczbą"
amount	Wartość musi być większa od 0	"Kwota musi być większa od 0"
description	Minimalna długość 5 znaków	"Opis musi mieć minimum 5 znaków"
categoryId	Pole nie może być puste	"Kategoria jest wymagana"
categoryId	Kategoria musi istnieć w liście	"Wybrana kategoria nie istnieje"

Kod walidacji:

```
const validateForm = () => {
  const newErrors: typeof errors = {};

  if (!date) {
    newErrors.date = "Data jest wymagana";
  } else if (new Date(date) > new Date()) {
    newErrors.date = "Data nie może być w przyszłości";
  }

  const amountNum = parseFloat(amount);
  if (!amount || isNaN(amountNum)) {
    newErrors.amount = "Kwota musi być liczbą";
  } else if (amountNum <= 0) {
    newErrors.amount = "Kwota musi być większa od 0";
  }

  if (description.trim().length < 5) {
    newErrors.description = "Opis musi mieć minimum 5 znaków";
  }

  if (!categoryId) {
    newErrors.categoryId = "Kategoria jest wymagana";
  } else if (!categories.find((c) => c.id === categoryId)) {
    newErrors.categoryId = "Wybrana kategoria nie istnieje";
  }

  setErrors(newErrors);
  return Object.keys(newErrors).length === 0;
};
```

4.2 Walidacja formularza kategorii

Lokalizacja: `src/components/categories/CategoryForm.tsx`

Pole	Warunek walidacji	Komunikat błędu
name	Minimalna długość 3 znaki	"Nazwa musi mieć minimum 3 znaki."

Kod walidacji:

```
const validateForm = () => {
  const newErrors: { name?: string } = {};

  if (name.trim().length < 3) {
    newErrors.name = "Nazwa musi miec minimum 3 znaki.";
  }

  setErrors(newErrors);
  return Object.keys(newErrors).length === 0;
};
```

5. Weryfikacja typow przekazywanych do komponentow

5.1 Lista typow props

Komponent	Interfejs	Liczba typow
Button	ButtonProps	6
InputField	InputFieldProps	10
SummaryCard	SummaryCardProps	4
TransactionForm	TransactionFormProps	4
TransactionItem	TransactionItemProps	5
CategoryForm	CategoryFormProps	3
CategoryItem	CategoryItemProps	3

5.2 Przyklady typow prostych

```
// string
label: string;
className: string;

// number
value: number;

// boolean
disabled: boolean;
required: boolean;

// function
onClick: () => void;
onChange: (value: string) => void;
onSubmit: (data: {...}) => void;
```

5.3 Przyklady typow zlozonych

Typ zlozony - TransactionFormProps

```
interface TransactionFormProps {  
  transaction?: Transaction | null; // obiekt lub null  
  (opcjonalny)  
  categories: Category[]; // tablica obiektow  
  onSubmit: (data: {  
    // funkcja z obiektem jako argument  
    date: string;  
    amount: number;  
    description: string;  
    categoryId: string;  
  }) => void;  
  onCancel: () => void;  
}
```

Typ zlozony - Union types

```
type: "button" | "submit" | "reset";  
variant: "primary" | "secondary" | "danger";  
type: "income" | "expense" | "balance";
```

6. Komponenty

6.1 Lista komponentow

Komponent	Typ	Lokalizacja
Button	Prezentacyjny	src/components/Button.tsx
InputField	Prezentacyjny	src/components/InputField.tsx
SummaryCard	Prezentacyjny	src/components/SummaryCard.tsx
TransactionItem	Prezentacyjny	src/components/transactions/TransactionItem.tsx
CategoryItem	Prezentacyjny	src/components/categories/CategoryItem.tsx
TransactionForm	Kontenerowy	src/components/transactions/TransactionForm.tsx
CategoryForm	Kontenerowy	src/components/categories/CategoryForm.tsx
Layout	Kontenerowy	src/layout/Layout.tsx
Dashboard	Kontenerowy	src/pages/Dashboard.tsx
TransactionList	Kontenerowy	src/pages/TransactionList.tsx
CategoryList	Kontenerowy	src/pages/CategoryList.tsx

6.2 Komponenty prezentacyjne (bez zmiennych stanu)

1. Button - przycisk z wariantami wizualnymi
2. InputField - pole tekstowe z etykieta i obsluga bledow
3. SummaryCard - karta podsumowania z wartoscia i ikona
4. TransactionItem - element listy transakcji

7. Dwukierunkowa komunikacja między komponentami

7.1 Przykład: TransactionList -> TransactionForm -> TransactionList

Lokalizacja: `src/pages/TransactionList.tsx`

Przepływ danych:

1. TransactionList przekazuje dane do TransactionForm (rodzic -> dziecko):

```
<TransactionForm
  transaction={editingTransaction} // dane do edycji
  categories={state.categories} // lista kategorii
  onSubmit={handleSubmit} // callback
  onCancel={handleCancel} // callback
/>
```

2. TransactionForm wywołuje callback z danymi (dziecko -> rodzic):

```
const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  if (validateForm()) {
    onSubmit({
      // wywołanie callbacka rodzica
      date,
      amount: parseFloat(amount),
      description: description.trim(),
      categoryId,
    });
  }
};
```

3. TransactionList odbiera dane i wykonuje akcje:

```
const handleSubmit = async (data: {...}) => {
  if (editingTransaction) {
    await actions.editTransaction(editingTransaction.id, data);
  } else {
    await actions.addTransaction(data);
  }
  setShowForm(false);
};
```

8. Komponenty reużywalne

8.1 Komponent Button

Lokalizacja: `src/components/Button.tsx`

Uzycia w aplikacji:

1. TransactionForm - przyciski "Dodaj transakcje" i "Anuluj"
2. CategoryForm - przyciski "Dodaj kategorie" i "Anuluj"
3. TransactionList - przycisk "Dodaj transakcje" i sortowanie
4. CategoryList - przycisk "Dodaj kategorie" i sortowanie
5. CategoryItem - przyciski edycji i usuwania
6. TransactionItem - przyciski akcji

8.2 Komponent InputField

Lokalizacja: `src/components/InputField.tsx`

Uzycia w aplikacji:

1. TransactionForm - pola: Data, Kwota, Opis
2. CategoryForm - pole: Nazwa kategorii

8.3 Komponent SummaryCard

Lokalizacja: `src/components/SummaryCard.tsx`

Uzycia w aplikacji:

1. Dashboard - karta "Przychody"
2. Dashboard - karta "Wydutki"
3. Dashboard - karta "Bilans"

9. Routing

Lokalizacja: `src/router.tsx`

9.1 Konfiguracja tras

Sciezka	Komponent	Opis
/	Dashboard	Strona glowna z podsumowaniem
/transactions	TransactionList	Lista transakcji
/transactions/add	TransactionAdd	Formularz dodawania transakcji
/transactions/:id	TransactionDetail	Szczegoly transakcji (parametr dynamiczny)
/categories	CategoryList	Lista kategorii
/categories/add	CategoryAdd	Formularz dodawania kategorii

9.2 Routing z parametrem

Lokalizacja: `src/pages/TransactionDetail.tsx`

Trasa: `/transactions/:id`

Uzycie parametru:

```
import { useParams } from "react-router";

export function TransactionDetail() {
  const { id } = useParams<{ id: string }>();
```

```
// ...
```

```
}
```

10. Architektura Flux

Lokalizacja: `src/flux/`

10.1 Skladniki architektury

Dispatcher

Plik: `src/flux/Dispatcher.ts`

Funkcje:

- `register(callback)` - rejestracja callbacka
- `dispatch(action)` - wysłanie akcji do wszystkich zarejestrowanych callbackow

Actions

Plik: `src/flux/actions.ts`

Lista akcji:

- `FETCH_CATEGORIES_REQUEST / SUCCESS / ERROR`
- `ADD_CATEGORY`
- `EDIT_CATEGORY`
- `DELETE_CATEGORY`
- `FETCH_TRANSACTIONS_REQUEST / SUCCESS / ERROR`
- `ADD_TRANSACTION`
- `EDIT_TRANSACTION`
- `DELETE_TRANSACTION`
- `SET_MONTH_FILTER`

Store

Plik: `src/flux/Store.ts`

Funkcje:

- `getState()` - zwraca aktualny stan
- `addChangeListener(callback)` - rejestracja listenera
- `removeChangeListener(callback)` - usunięcie listenera
- `emitChange()` - powiadomienie o zmianie stanu

10.2 Przepływ danych

1. Komponent wywołuje akcje:

```
actions.fetchTransactions();
```

2. Akcja wykonuje zadanie HTTP i wysyła do Dispatchera:

```
const transactions = await API.transactions.all();
dispatcher.dispatch({
  type: ActionTypes.FETCH_TRANSACTIONS_SUCCESS,
```

```
    payload: transactions,  
  });
```

3. Dispatcher przekazuje akcje do Store:

```
dispatch(action: any): void {  
  this.callbacks.forEach((callback) => callback(action));  
}
```

4. Store aktualizuje stan i emituje zmianę:

```
case ActionTypes.FETCH_TRANSACTIONS_SUCCESS:  
  this.state.transactions = action.payload;  
  this.state.loading.transactions = false;  
  this.emitChange();  
  break;
```

5. Komponent reaguje na zmianę:

```
useEffect(() => {  
  const handleChange = () => setState(store.getState());  
  store.addChangeListener(handleChange);  
  return () => store.removeChangeListener(handleChange);  
}, []);
```

11. Podsumowanie punktacji

Wymaganie	Realizacja	Punkty
Struktura danych dwupoziomowa	Transaction -> Category (relacja przez categoryId)	1 pkt
Dodawanie transakcji	TransactionForm, actions.addTransaction	0.5 pkt
Dodawanie kategorii	CategoryForm, actions.addCategory	0.5 pkt
Edycja transakcji	TransactionForm, actions.editTransaction	0.5 pkt
Edycja kategorii	CategoryForm, actions.editCategory	0.5 pkt
Usuwanie transakcji	actions.deleteTransaction	0.5 pkt
Usuwanie kategorii	actions.deleteCategory	0.5 pkt
Sortowanie transakcji (4 kierunki)	date-desc, date-asc, amount-desc, amount-asc	0.5 pkt

Sortowanie kategorii (2 kierunki)	asc, desc	0.5 pkt
Weryfikacja typow (min. 5)	ButtonProps, InputFieldProps, SummaryCardProps, TransactionFormProps, CategoryFormProps, TransactionItemProps, CategoryItemProps	1.5 pkt
Komponenty (min. 5)	Button, InputField, SummaryCard, TransactionItem, CategoryItem, TransactionForm, CategoryForm, Layout	4 pkt
Dwukierunkowa komunikacja	TransactionList <-> TransactionForm, CategoryList <-> CategoryForm	1 pkt
Komponenty reuzywalne	Button (6+ uzyc), InputField (4+ uzyc), SummaryCard (3 uzycia)	2 pkt
Walidacja formularzy	TransactionForm (5 regul), CategoryForm (1 regula)	1.5 pkt
Operacje HTTP (4 rodzaje)	GET, POST, PATCH, DELETE	2 pkt
Routing z parametrem	/transactions/:id	1.5 pkt
Architektura Flux	Dispatcher, Store, Actions	2 pkt